

**Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ**

**Лабораторная работа №6
По дисциплине «Современные платформы программирования»**

Выполнил:
Студент 3 курса
Группы ПО-12
Пашкевич В. В.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета *pytest*.

Задание 1

Написание тестов для мини-библиотеки покупок (*shopping.py*)

shopping.py:

```
import requests

class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append({"name": name, "price": price})

    def total(self):
        return sum(item["price"] for item in self.items)

    def apply_discount(self, percent):
        if not 0 <= percent <= 100:
            raise ValueError("Discount must be between 0 and 100")

        current_total = self.total()
        return current_total * (1 - percent / 100)

# Функции для логирования и купонов

def log_purchase(item):
    requests.post("https://example.com/log", json=item, timeout=5)

def apply_coupon(cart, coupon_code, custom_coupons=None):
    coupons = custom_coupons if custom_coupons is not None else {"SAVE10":
10, "HALF": 50}
    if coupon_code in coupons:
        return cart.apply_discount(coupons[coupon_code])
    raise ValueError("Invalid coupon")
```

test_cart.py

```
from unittest.mock import patch
import pytest
from shopping import Cart, apply_coupon
from shopping import log_purchase
# pylint: disable=redefined-outer-name

@pytest.fixture
def empty_cart():
    return Cart()

def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
```

```

def test_negative_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("Apple", -1.0)

def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)
    assert empty_cart.total() == 15.0

@pytest.mark.parametrize("discount, expected", [
    (0, 100),
    (50, 50),
    (100, 0),
])
def test_apply_discount(empty_cart, discount, expected):
    empty_cart.add_item("Item", 100)
    assert empty_cart.apply_discount(discount) == expected

@pytest.mark.parametrize("invalid_discount", [-10, 110])
def test_apply_discount_error(empty_cart, invalid_discount):
    with pytest.raises(ValueError):
        empty_cart.apply_discount(invalid_discount)

# Тест логирования с моком
def test_log_purchase():
    with patch('requests.post') as mocked_post:
        item = {"name": "Apple", "price": 10.0}
        log_purchase(item)
        mocked_post.assert_called_once_with("https://example.com/log",
        json=item)

# Тест купонов с patch.dict
def test_apply_coupon(empty_cart):
    empty_cart.add_item("Item", 100)
    # Передаем словарь прямо в функцию
    my_coupons = {"TEST": 20}
    assert apply_coupon(empty_cart, "TEST", custom_coupons=my_coupons) == 80

```

```

Run pytest in test_cart.py x
Test Results 0 ms 10 tests passed 10 tests total, 0 ms

===== test session starts =====
collecting ... collected 10 items

reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_add_item PASSED [ 10%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_negative_price PASSED [ 20%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_total PASSED [ 30%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_apply_discount[0-100] PASSED [ 40%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_apply_discount[50-50] PASSED [ 50%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_apply_discount[100-0] PASSED [ 60%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_apply_discount_error[-10] PASSED [ 70%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_apply_discount_error[110] PASSED [ 80%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_log_purchase PASSED [ 90%]
reports/Pashkevich_V_V/LAB06/src/test_cart.py::test_apply_coupon PASSED [100%]

===== 10 passed in 0.22s =====

Process finished with exit code 0

```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случаи, а также варианты, когда

может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

refactored_lab1.py:

```
def sum_squares_negatives(numbers):
    return sum(x**2 for x in numbers if x < 0)

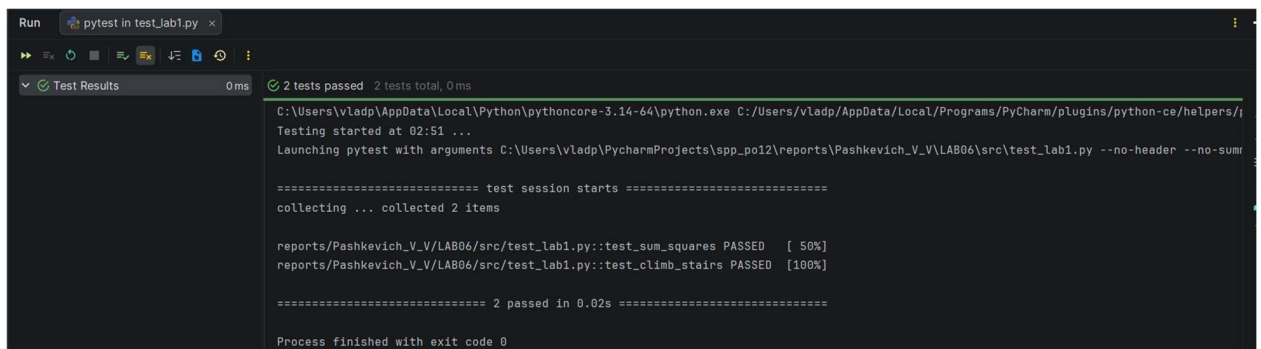
def count_climbing_ways(n):
    if n <= 0:
        raise ValueError("Must be positive")
    if n == 1:
        return 1
    if n == 2:
        return 2
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

test_lab1.py:

```
import pytest
from refactored_lab1 import sum_squares_negatives, count_climbing_ways

def test_sum_squares():
    assert sum_squares_negatives([-1, -2, 3]) == 5
    assert sum_squares_negatives([1, 2]) == 0

def test_climb_stairs():
    assert count_climbing_ways(3) == 3
    assert count_climbing_ways(4) == 5
    with pytest.raises(ValueError):
        count_climbing_ways(0)
```



The screenshot shows the 'Run' window in PyCharm with the title 'pytest in test_lab1.py'. The 'Test Results' tab is active, displaying '2 tests passed' and '2 tests total, 0ms'. The command line shows the execution of 'C:\Users\vladp\AppData\Local\Python\pythoncore-3.14-64\python.exe C:\Users\vladp\AppData\Local\Programs\PyCharm\plugins\python-ce\helpers\...'. The test session output includes 'collecting ... collected 2 items', followed by 'reports\Pashkevich_V_V\LAB06\src\test_lab1.py::test_sum_squares PASSED [50%]' and 'reports\Pashkevich_V_V\LAB06\src\test_lab1.py::test_climb_stairs PASSED [100%]'. The session concludes with '2 passed in 0.02s' and 'Process finished with exit code 0'.

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Вариант 3

Напишите метод **String keep(String str, String pattern)** который оставляет в первой строке все символы, которые присутствуют во второй.

Спецификация метода:

```
keep (None , None ) = TypeError
keep (None, *) = None
keep ("", *) = ""
keep (* , None ) = ""
keep (* , "") = ""
keep (" hello ", "hl") = " hll "
keep (" hello ", "le") = " ell "
```

zadanie3.py:

```
def keep(str_val, pattern):
    # Реализация логики согласно спецификации
    if str_val is None or pattern is None:
        raise TypeError("TypeError")

    # Логика: оставляем только те символы из str_val, которые есть в pattern
    return "".join([char for char in str_val if char in pattern])
```

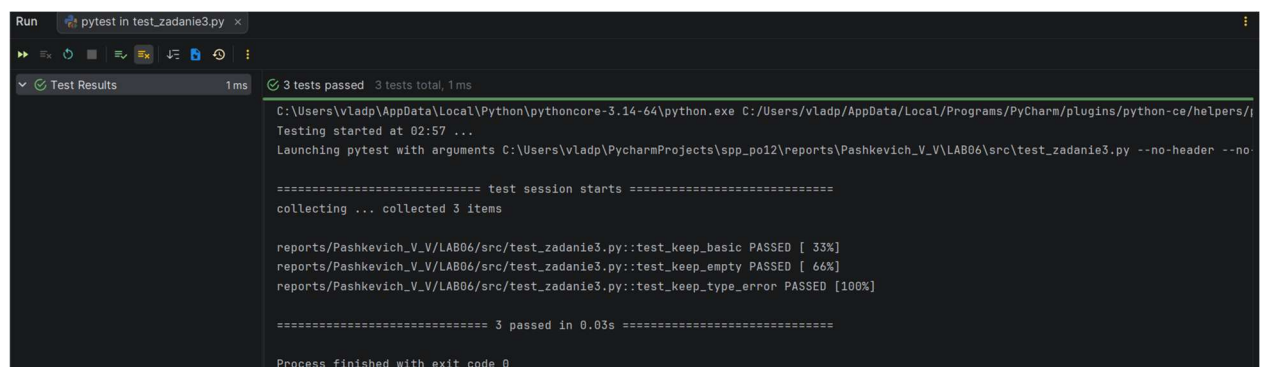
test_zadanie3.py:

```
import pytest
from zadanie3 import keep

def test_keep_basic():
    # Чтобы получить " hll ", нужно разрешить и 'h', и 'l', и ' ' (пробел)
    assert keep(" hello ", "hl ") == " hll "

def test_keep_empty():
    assert keep("", "abc") == ""
    assert keep("abc", "") == ""

def test_keep_type_error():
    with pytest.raises(TypeError):
        keep(None, "abc")
    with pytest.raises(TypeError):
        keep("abc", None)
```



```
Run pytest in test_zadanie3.py
Test Results 1 ms 3 tests passed 3 tests total, 1 ms
C:\Users\vladp\AppData\Local\Python\pythoncore-3.14-64\python.exe C:\Users\vladp\AppData\Local\Programs\PyCharm\plugins\python-ce\helpers/i
Testing started at 02:57 ...
Launching pytest with arguments C:\Users\vladp\PycharmProjects\spp_po12\reports\Pashkevich_V_V\LAB06\src\test_zadanie3.py --no-header --no-
===== test session starts =====
collecting ... collected 3 items
reports\Pashkevich_V_V\LAB06\src\test_zadanie3.py::test_keep_basic PASSED [ 33%]
reports\Pashkevich_V_V\LAB06\src\test_zadanie3.py::test_keep_empty PASSED [ 66%]
reports\Pashkevich_V_V\LAB06\src\test_zadanie3.py::test_keep_type_error PASSED [100%]
===== 3 passed in 0.03s =====
Process finished with exit code 0
```

Вывод: освоил приемы тестирования кода на примере использования пакета *pytest*.